

intellementaINTERNAL STANDARD

Technical Documentation Standard

How we write technical documentation at Intellementa

This is the standard every engineer follows when documenting a system. Its job is simple: anyone on the team should be able to open your document and understand how the system works without having to track you down. Follow it for architecture documents, module specifications, milestone reports, and handover documents.

When to use this

Use this standard whenever you write documentation that describes how a system works — its design, how it is configured, how it runs, and how someone would maintain or extend it. It does not apply to user-facing product help, sales material, or marketing content.

Write the document as if the reader is a capable engineer who has never seen the system and cannot ask you questions. If your document passes that test, it passes this standard.

The sections every document must have

Every technical document contains the fifteen sections below, in this order. Keep the order even if a section is short. If a section genuinely does not apply, leave the heading in and write one sentence saying why.

#	Section	What goes in it
1	Project Overview	What the system does, who it is for, and what it is not.
2	System Architecture	The big picture and the main workflow.
3	Technology Stack	Languages, libraries, and services, with why each is used.
4	Repository Structure	The folder and file layout.
5	Configuration	Environment variables and settings.
6	Shared Infrastructure	Shared utilities and the core data models.
7	Core Component	The part that ties everything together.
8	Components	Each major module, documented the same way (see below).
9	Data Layer	Where data comes from, its format, and where it is stored.
10	Frontend / Entry Points	How the system is actually run.
11	Logging & Artifacts	What gets written to disk and how long it is kept.
12	Data Flow	How information moves between the parts.
13	Remaining Work	What is not built yet, and what it depends on.
14	Known Limitations	What is fragile or unfinished today.
15	Getting Started	How a new developer gets the system running.

How to document a component

Section 8 is the part of the document people read most, so it has to be predictable. Document every component the same way, in the same order. Once a reader has read one, they know exactly where to look in all the others.

- **Location** — the file or folder where the component lives.
- **Role** — one short paragraph: what it does and why it exists.
- **Inputs** — a table of what it takes in: each field, its type, and a description.
- **Processing** — a numbered list of what it does, step by step.
- **Outputs** — what it produces, pointing back to the data models in Section 6.
- **Artifacts** — any files it writes to disk, with their paths and contents.

Keep it uniform. Do not reinvent this layout per component. The most common reason a document becomes hard to read is that each component is written up differently.

Formatting

- Put structured information in tables — environment variables, data fields, the tech stack, and so on.
- Use a numbered list for steps that happen in order, and a bulleted list for everything else.
- Put code, file paths, commands, and names in a monospaced font, and always explain what a code block is for.
- Keep headings to two levels (1, 1.1). Use a third level only when you really need it, never deeper.

How to write

Write in plain, direct language. Skip marketing words like **powerful**, **seamless**, or **world-class** — they tell the reader nothing. Spell out an acronym the first time you use it, then use the short form.

Be honest about what is not finished. Sections 13 and 14 — Remaining Work and Known Limitations — are the two people are most tempted to skip, and they are the two that matter most to whoever picks up the system after you. List what is half-built, what is fragile, and what depends on something that does not exist yet. A document that hides its gaps wastes the next person's time and is not complete.

Write Section 15 for the person joining next week. Getting Started should let a new developer install, run, and find their way around the system on their own. The single most useful thing you can give them is the order in which to read the source files — write that part last, once you can see the whole system clearly.

Before you share it

Run through this before you send a document round:

- ☐ All fifteen sections are present, in order.
- ☐ Every component in Section 8 follows the same six-part layout.
- ☐ Remaining Work and Known Limitations are filled in honestly.
- ☐ A new joiner could get the system running from Section 15 alone.
- ☐ Structured information is in tables; steps are numbered.

- ☐ No marketing language, and every acronym is spelled out once.

Decision rule. If every box is ticked, the document is ready. If you are unsure whether something belongs, ask before you publish rather than after.